Servlet-based applications in Polar Bookshop: Catalog Service and Config Service. Reactive applications are already configured with fewer threads, since they are much more resource-efficient than their imperative counterparts. For that reason, we haven't customized the thread count for Edge Service, Order Service, or Dispatcher Service.

NOTE The Paketo team is working on extending the Java Memory Calculator to provide a low-profile mode, which will be helpful when working locally or on low-volume applications. In the future, it will be possible to control the memory configuration mode via a flag rather than having to tweak the individual parameters. You can find more information about this feature on the GitHub project for Paketo Buildpacks (http://mng.bz/5Q87).

The JVM has two main memory areas: heap and non-heap. The Calculator focuses on computing values for the different non-heap memory parts according to a specific formula. The remaining memory resources are assigned to the heap. If the default configuration is not good enough, you can customize it as you prefer. For example, I experienced some memory issues with an imperative application handling session management with Redis. It required more direct memory than was configured by default. In that case, I used the standard `-XX:MaxDirectMemorySize=50M` JVM setting via the `JAVA_TOOL_OPTIONS` environment variable and increased the maximum size for the direct memory from 10 MB to 50 MB. If you customize the size of a specific memory region, the Calculator will adjust the allocation of the remaining areas accordingly.

NOTE Memory handling in the JVM is a fascinating topic that would require its own book to fully cover. Therefore, I won't go into details regarding how to configure it.

Since we are configuring deployments for production, let's update the thread count for Catalog Service using a more suitable number like `100`. In a real-world scenario, I would recommend starting with the default value of `250` as a baseline. For Polar Bookshop, I'm trying to compromise between showing what an actual production deployment would look like and